

Translated

From: e-maxx.ru

Minimum spanning tree - Kruskal with Disjoint Set Union

For an explanation of the MST problem and the Kruskal algorithm, first see the [main article on Kruskal's algorithm](#).

In this article we will consider the data structure "[Disjoint Set Union](#)" for implementing Kruskal's algorithm, which will allow the algorithm to achieve the time complexity of $O(M \log N)$.

Description

Just as in the simple version of the Kruskal algorithm, we sort all the edges of the graph in non-decreasing order of weights. Then put each vertex in its own tree (i.e. its set) via calls to the `make_set` function - it will take a total of $O(N)$. We iterate through all the edges (in sorted order) and for each edge determine whether the ends belong to different trees (with two `find_set` calls in $O(1)$ each). Finally, we need to perform the union of the two trees (sets), for which the DSU `union_sets` function will be called - also in $O(1)$. So we get the total time complexity of $O(M \log N + N + M) = O(M \log N)$.

Implementation

Here is an implementation of Kruskal's algorithm with Union by Rank.

```
vector<int> parent, rank;

void make_set(int v) {
    parent[v] = v;
    rank[v] = 0;
}

int find_set(int v) {
    if (v == parent[v])
        return v;
    return parent[v] = find_set(parent[v]);
}

void union_sets(int a, int b) {
    a = find_set(a);
```